PHISHGUARD: MOBILE PHISHING DETECTION APPLICATION

Students: Magetz Radu & Teodor Barbuceanu
Subject: Security of mobile devices

1. Architectural System Design

The PhishGuard framework utilizes a split-tier architecture distributed between a high-efficiency native mobile client layer and an asynchronous cloud backend telemetry/intelligence infrastructure. This topology ensures that intensive, continuous offline deep-learning cycles and massive threat aggregation remain segregated from the mobile device, preserving battery integrity, minimizing network overhead, and enforcing strict user data privacy. The on-device runtime components operate autonomously, guaranteeing continuous threat isolation capabilities even during periods of network degradation or complete disconnection.

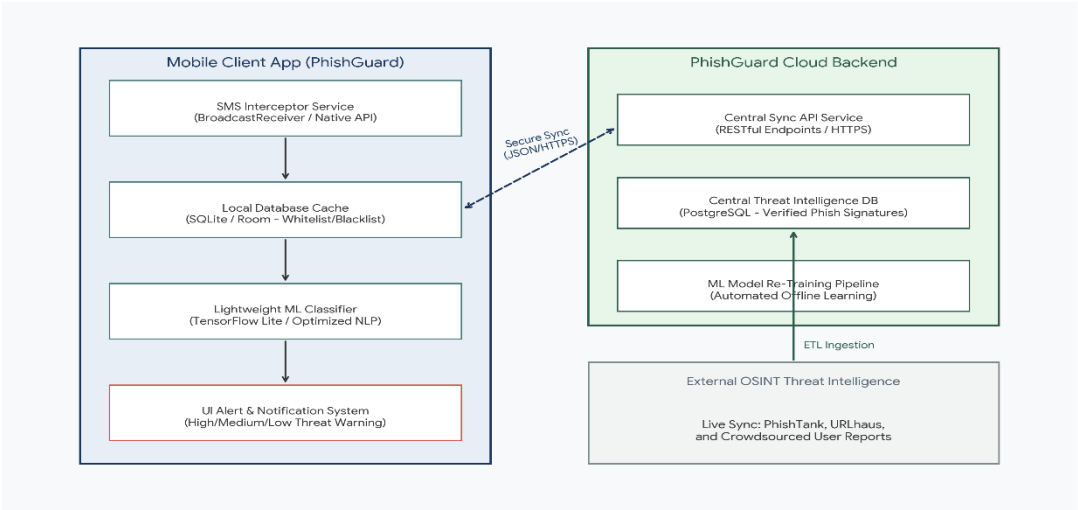

Figure 1: PhishGuard System Architecture—Decoupled Client-Server Micro-components and Data Topologies.

As mapped above, the system components are partitioned into clear structural layers:

- Mobile Application Layer: Contain the native background interceptor loop, the local SQLite/Room database caching tier containing high-confidence threat identifiers, the quantized TensorFlow Lite machine learning engine, and the visual Alert Notification UI.
- Cloud Backend Server Layer: Handles secure HTTPS-bound synchronization requests, hosts the relational database layer storing historical telemetry, and executes scheduled Extract-Transform-Load (ETL) jobs connecting to active global Open Source Intelligence (OSINT) streams (e.g., PhishTank, URLhaus) to continuously refine threat signatures.

2. Component Control and Data Flow Analysis

To achieve non-invasive, deterministic threat detection, execution logic moves sequentially through standardized system stages. Data integrity is prioritized during ingestion, ensuring that message fields are sanitized, isolated, and rapidly parsed. Figure 2 establishes the lifecycle of an incoming SMS vector through the application sub-systems:

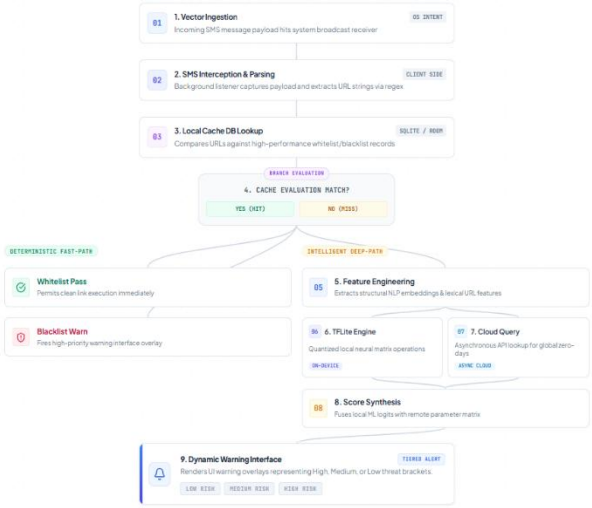

Figure 2: PhishGuard Synchronous and Asynchronous Operational Logic and Classification Flow Chart.

The PhishGuard execution pipeline begins with vector ingestion, where an incoming SMS message payload lands on the mobile handset via cellular transceivers, activating the underlying operating system broadcast routine. This is immediately followed by interception and parsing, a phase where PhishGuard's low-overhead background listener captures the intent payload, extracts raw textual content arrays and string metadata, and passes them to a localized regular-expression parsing utility to isolate embedded URL substrings.

3. Comprehensive Technical Planning & Implementation Roadmap

Implementing the PhishGuard application requires adherence to modern software engineering workflows. The development lifecycle follows strict rigorous automation criteria, including automated continuous integration (CI) environments, mandatory coding standard linters, detailed multi-peer branch reviews, and iterative validation milestones. The sequential execution roadmap is formalized in Table 1.

Phase ID	Operational Milestone	Technical Steps & Code Operations	Quality Gate
P1	**Environment & CI/CD Setup**	• Initialize central Git repository with main/develop branches. • Configure Git branch protection rules (require PR approvals, status checks, linear history).	100% linter compliance; passing CI pipeline on null commits.

Phase ID	Operational Milestone	Technical Steps & Code Operations	Quality Gate
P2	**Data Sourcing & Core ML Engineering**	• Aggregate public historical smishing and URL corpuses (SMS Spam Collection, PhishTank dumps). • Execute Python-based tokenization, vectorization, and feature extraction via Scikit-Learn / Pandas. • Train classification baselines (RandomForest, XGBoost, and an LSTM variant for NLP body analysis).	Validation accuracy score ≥ 95%; export size under 15MB limit.
P3	**Native Mobile Core Implementation**	• Implement background Android BroadcastReceiver or iOS Notification Extension system. • Code the local filesystem binary importer to load and read the compiled TFLite model safely. • Construct the responsive UI views mapping threats into striking warning categories (High/Medium/Low).	Zero main-thread blocking during interception loop execution.
P4	**Component Integration & Code Review**	• Bind the mobile cache miss routine to trigger the embedded TFLite classifier dynamically. • Hook mobile sync schedulers into background workers to call Cloud API endpoints on connection..	Critical architectural code reduced.

4. Short summary

Following the successful implementation, the PhishGuard application enforces long-term stability via automated analytical pipelines. As threats evolve, the local ML classifier requires structured monitoring to detect degradation in model efficacy. User submissions regarding anomalous, unclassified smishing alerts are compiled on the central repository backend, aggregated weekly, and pushed back into the offline training infrastructure. The resultant newly-tuned models are then optimized and distributed to the global active mobile installation base via incremental background updates, ensuring that the system target of a 95%+ classification accuracy benchmark is continuously maintained while ensuring minimum false-positives and maintaining an institutional, academic rigor throughout the lifecycle.